

Unit II: Logical Instructions and Control/Branch Instructions

Logical Instructions, String manipulation instructions, Flag manipulation and Processor control instructions, Illustration of these instructions with example programs, control and branch instructions. Assembler Directives and Operators, Assembly Language Programming and example programs

Logical Instructions

Mnemonic	Meaning	Format	Operation
AND	Logical AND	AND D, S	$(S) \cdot (D) \rightarrow (D)$
OR	Logical Inclusive —OR	OR D, S	$(S) + (D) \rightarrow (D)$
XOR	Logical Exclusive —OR	XOR D, S	$(S) \oplus (D) \rightarrow (D)$
NOT	Logical NOT	NOT D	$(\overline{D}) \rightarrow (D)$

AND – AND Destination, Source

- This instruction ANDs each bit in a source byte or word with the same numbered bit in a destination byte or word.
- The result is put in the specified destination. The content of the specified source is not changed.
- The source can be an immediate number, the content of a register, or the content of a memory location.
- The destination can be a register or a memory location.
- The source and the destination cannot both be memory locations.
- CF and OF are both 0 after AND.
- PF, SF, and ZF are updated by the AND instruction. AF is undefined. PF has meaning only for an 8-bit operand.

Examples

- AND CX, [SI] ;AND word in DS at offset [SI] with word in CX register;
- Result in CX register
- AND BH, CL ;AND byte in CL with byte in BH; Result in BH
- AND BX, 00FFH; 00FFH Masks upper byte, leaves lower byte unchanged
- Examples: If BX=5544h, After AND BX, 00FFh, BX=??
- If SI=2000h, DS=1000h, Content of memory addressed by SI=1234h, after ADD BX, [SI], the content of BX=???
- Status flags CF, OF, PF, SF, and ZF are updated by the AND instruction.
- AF is undefined

- If SI=2000h, DS=1000h, Content of memory addressed by SI=1234h, after ADD BX, [SI], the content of BX=???

OR – OR Destination, Source

- This instruction ORs each bit in a source byte or word with the same numbered bit in a destination byte or word.
- The result is put in the specified destination.
- The content of the specified source is not changed.
- The source can be an immediate number, the content of a register, or the content of a memory location.
- The destination can be a register or a memory location.
- The source and destination cannot both be memory locations.
- CF and OF are both 0 after OR.
- PF, SF, and ZF are updated by the OR instruction.
- AF is undefined.
- PF has meaning only for an 8-bit operand.

Examples

- OR AH, CL ;CL ORed with AH, result in AH, CL not changed
- OR BP, SI; SI ORed with BP, result in BP, SI not changed
- OR SI, BP; BP ORed with SI, result in SI, BP not changed
- OR BL, 80H; BL ORed with immediate number 80H; sets MSB of BL to 1
- OR CX, TABLE [SI]; CX ORed with word from effective address TABLE [SI], Content of memory is not changed.

Examples

AND : Ex- AND AX,0008H

Let AX is 3FoFH

0 0 1 1	1 1 1 1	0 0 0 0	1 1 1 1	= 3FoF H {AX}
↓ ↓ ↓ ↓	↓ ↓ ↓ ↓	↓	↓	AND
0 0 0 0	0 0 0 0	0 0 0 0	1 0 0 0	= 0008

0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 = 0008 H {AX}

OR : Ex- OR AX,0098H

Let AX is 3FoFH

0 0 1 1	1 1 1 1	0 0 0 0	1 1 1 1	= 3FoF H {AX}
↓	↓	↓	↓	OR
0 0 0 0	0 0 0 0	0 0 0 0	1 0 0 0	= 0098

0 0 1 1 1 1 1 1 1 0 0 1 1 1 1 1 = 3F9F H {AX}

XOR – XOR Destination, Source

- This instruction Exclusive-ORs each bit in a source byte or word with the same numbered bit in a destination byte or word.
- The result is put in the specified destination. The content of the specified source is not changed.
- The source can be an immediate number, the content of a register, or the content of a memory location.
- The destination can be a register or a memory location.
- The source and destination cannot both be memory locations. CF and OF are both 0 after XOR.
- PF, SF, and ZF are updated. PF has meaning only for an 8-bit operand. AF is undefined.

Examples

- XOR CL, BH Byte in BH exclusive-ORed with byte in CL, Result in CL. BH not changed.
- XOR BP, DI Word in DI exclusive-ORed with word in BP, Result in BP. DI not changed.
- XOR WORD PTR [BX], 00FFH ;Exclusive-OR immediate number 00FFH with word at offset [BX] in the data segment.
- Result in memory location [BX]

NOT – NOT Destination

- The NOT instruction inverts each bit (forms the 1's complement) of a byte or word in the specified destination. The destination can be a register or a memory location. This instruction does not affect any flag.
- NOT BX Complement content of BX register
- NOT BYTE PTR [BX] Complement memory byte at offset [BX] in data segment
- Ex: If AX = 8345h = 1000 0011 0100 0101
- After NOT AX, AX = 0111 1100 1011 1010

NEG – NEG Destination

- This instruction replaces the number in a destination with its 2's complement. The destination can be a register or a memory location. It gives the same result as the *invert each bit and add one* algorithm. The NEG instruction updates AF, AF, PF, ZF, and OF.
- NEG AL Replace number in AL with its 2's complement
- NEG BX Replace number in BX with its 2's complement
- NEG BYTE PTR [BX] Replace byte at offset BX in DX with its 2's complement
- NEG WORD PTR [BP] Replace word at offset BP in SS with its 2's complement

CMP – CMP Destination, Source

- This instruction compares a byte / word in the specified source with a byte / word in the specified destination.
- The source can be an immediate number, a register, or a memory location.
- The destination can be a register or a memory location.
- However, the source and the destination cannot both be memory locations.
- The comparison is actually done by subtracting the source byte or word from the destination byte or word.
- The source and the destination are not changed, but the flags are set to indicate the results of the comparison.
- AF, OF, SF, ZF, PF, and CF are updated by the CMP instruction.
-

Examples

For the instruction `CMP CX, BX`, the values of CF, ZF, and SF will be as follows:

	CF	ZF	SF
--	----	----	----

- `CX = BX` 0 1 0 Result of subtraction is 0
- `CX > BX` 0 0 0 o borrow required, so CF = 0
- `CX < BX` 1 0 1 Subtraction requires borrow, so CF = 1
- `CMP AL, 01H` ; Compare immediate number 01H with byte in AL
- `CMP BH, CL`; Compare byte in CL with byte in BH
- `CMP CX, TEMP` ;Compare word in DS at displacement TEMP with word at CX
- `CMP PRICES [BX], 49H` ;Compare immediate number 49H with byte at offset [BX] in array PRICES

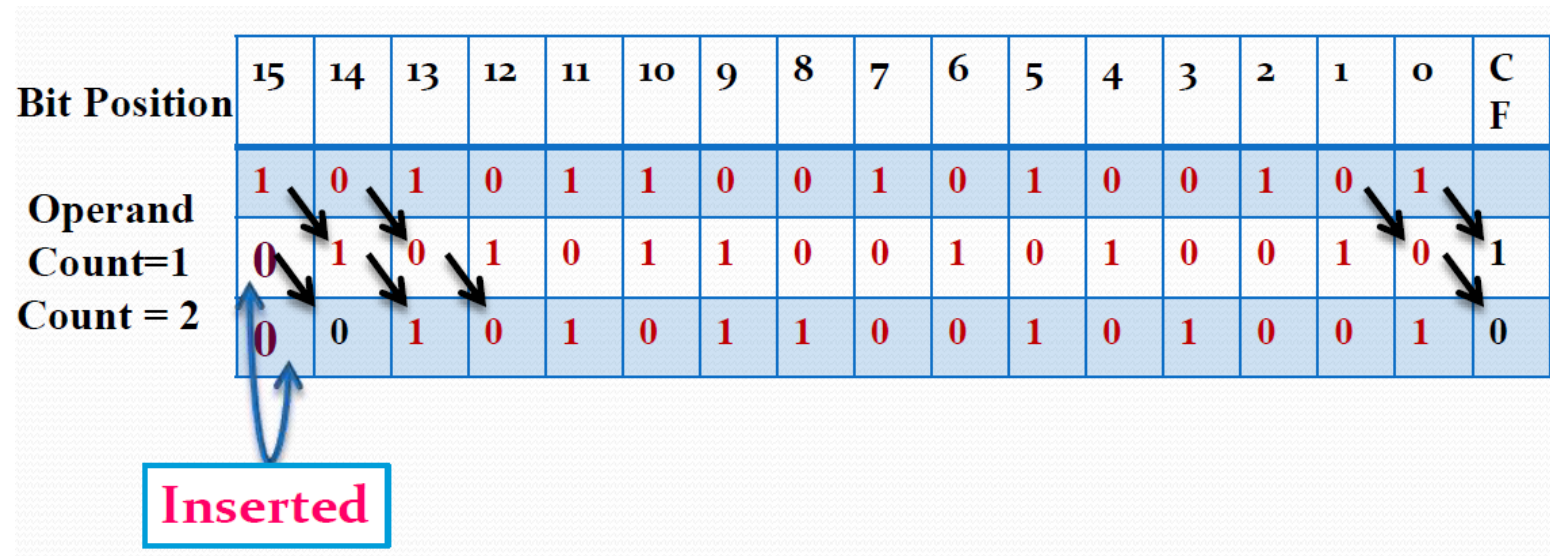
TEST – TEST Destination, Source

- This instruction ANDs the byte / word in the specified source with the byte / word in the specified destination. Flags are updated, but neither operand is changed. The test instruction is often used to set flags before a Conditional jump instruction.
- The source can be an immediate number, the content of a register, or the content of a memory location. The destination can be a register or a memory location. The source and the destination cannot both be memory locations. CF and OF are both 0's after TEST. PF, SF and ZF will be updated to show the results of the destination. AF is be undefined.
- TEST AL, BH AND BH with AL. No result stored; Update PF, SF, ZF.
- TEST CX, 0001H AND CX with immediate number 0001H;
- No result stored; Update PF, SF, ZF
- TEST BP, [BX][DI] AND word are offset [BX][DI] in DS with word in BP.
- No result stored. Update PF, SF, and ZF

Shift & Rotate instruction

1. Shift Logical Right SHR : SHR Des, Count

- It shift bits of byte or word right, by count.
- It puts zero(s) in MSBs.
- LSB is shifted into carry flag.
- If the number of bits desired to be shifted is 1, then the immediate number 1 can be written in Count.
- However, if the number of bits to be shifted is more than 1, then the count is put in CL register.



SHL Des, Count

- It shift bits of byte or word left, by count.
- It puts zero(s) in LSBs.
- MSB is shifted into carry flag.
- If the number of bits desired to be shifted is 1, then the immediate number 1 can be written in Count.
- However, if the number of bits to be shifted is more than 1, then the count is put in CL register.

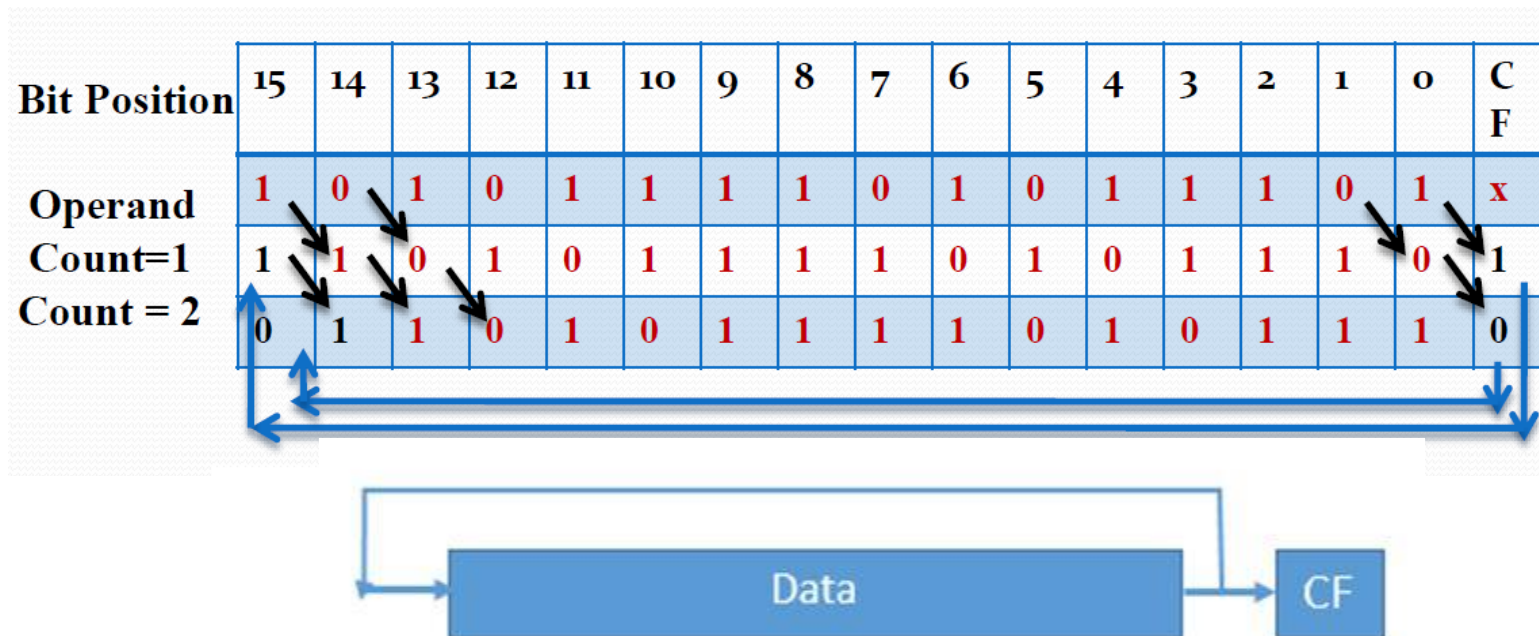
Bit Position

	CF	1	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Operand		5															
Count=1		1	0	1	0	1	1	0	0	1	0	1	0	0	1	0	1
Count = 2	1	0	1	0	1	1	0	0	1	0	1	0	0	1	0	1	0
	0	1	0	1	1	0	0	1	0	1	0	0	1	0	1	0	0

Inserte

ROR Des, Count (Rotate right without carry)

- It rotates bits of byte or word right, by count.
- LSB is transferred to MSB and also to CF.
- If the number of bits desired to be shifted is 1, then the immediate number 1 can be written in Count.
- However, if the number of bits to be shifted is more than 1, then the count is put in CL register.



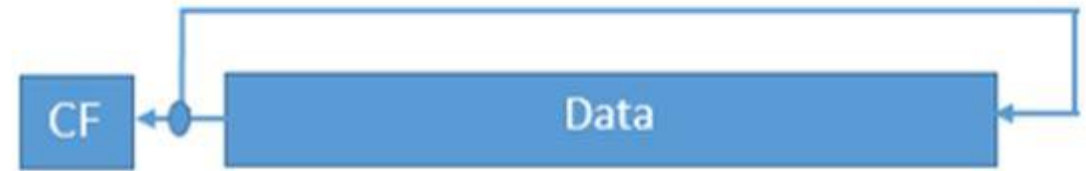
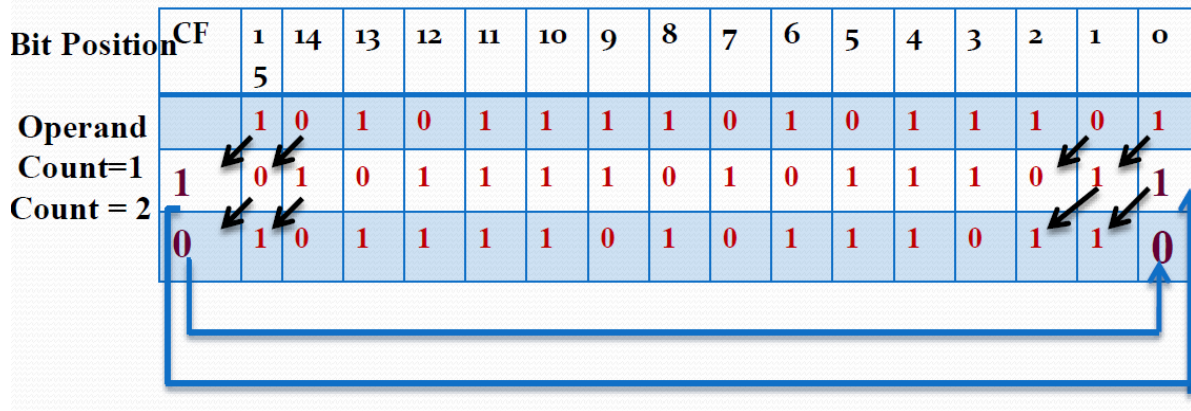
Examples :

```
ROR  CX,  1           ; Rotated word in CX one bit position
                        ; left, LSB to MSB and CF.
MOV  CL,  03H         ; Load number of bits to rotate in CL.
ROR  BL,  CL          ; Rotate BL three positions.
```

- If BL=9Bh, What is the output when the above mentioned instructions are executed?

ROL Des, Count

- It rotates bits of byte or word left, by count.
- MSB is transferred to LSB and also to CF.
- If the number of bits desired to be shifted is 1, then the immediate number 1 can be written in Count.
- However, if the number of bits to be shifted is more than 1, then the count is put in CL register.



Examples :

```
ROL  CX,  1      ; Word in CX one bit position left, MSB to
                  ; LSB and CF
MOV  CL,  03H    ; Load desired number of bits to rotate in CL.
ROL  BL,  CL      ; Rotate BL three positions.
```

If BL=9Bh, What is the output when the above mentioned instructions are executed?

Flag Manipulation & Processing(Machine) Control Instructions

- Controls the functioning of available hardware inside the processor chip.
- F.M. Instructions directly modify some of the flags of 8086

CLC	Clear Carry Flag
CMC	Complement Carry Flag
STF	Set Carry Flag
CLD	Clear directional flag
STD	Set Directional Flag
CLI	Clear Interrupt flag
STI	Set Interrupt Flag

Program Execution Transfer Instructions

- These instructions cause change in the sequence of the execution of instruction.
- This change can be through a condition or sometimes unconditional.
- The conditions are represented by flags.

Program Execution Transfer Instructions

- These instructions cause change in the sequence of the execution of instruction.
- This change can be through a condition or sometimes unconditional.
- The conditions are represented by flags.

Program Execution Transfer Instructions

CALL Des

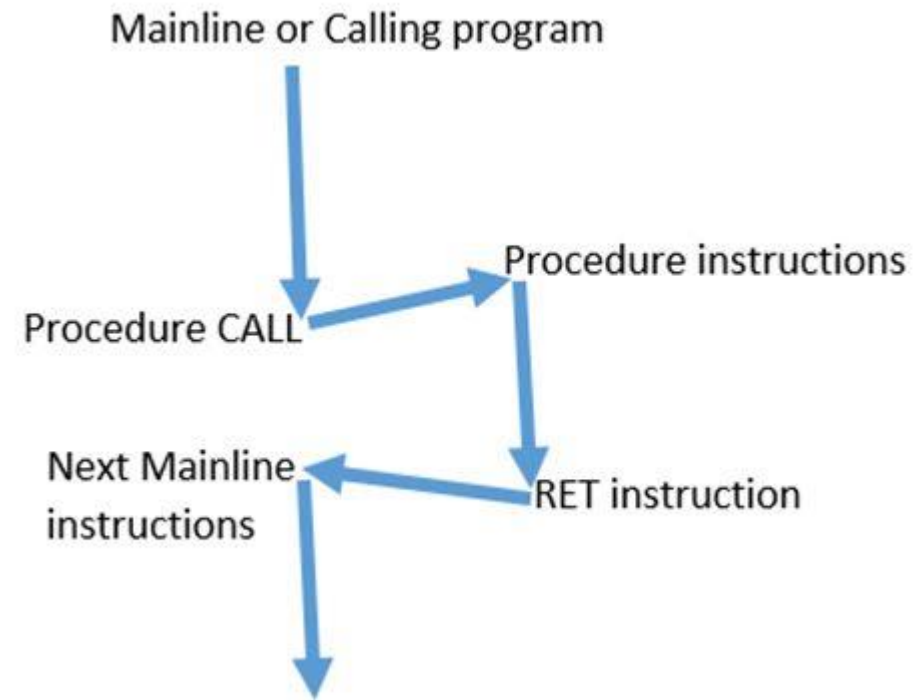
- This instruction is used to call a subroutine or function or procedure.
- The address of next instruction after CALL is saved onto stack.
- On execution, it stores the incremented IP & CS onto the stack and loads the CS & IP registers with segment and offset addresses of the procedure to be called

Ex: CALL Sub_program name

RET

- It returns the control from procedure to calling program.
- Every CALL instruction should have a RET.
- When it is executed, the previously stored content of IP and CS along with Flags are retrieved into the CS, IP and Flag registers from the stack and execution of the main program continues further

Illustration of CALL and RET instruction



Program Execution Transfer Instructions

- **JMP Des:**
- This instruction is used for unconditional jump from one place to another.
- **Jxx Des (Conditional Jump):**
- All the conditional jumps follow some conditional statements or any instruction that affects the flag.

Conditional Jump Table

Mnemonic	Description	Clock Cycles	No. of bytes	Example / Remarks
JMP Label	Jump unconditionally; Can be <i>Near</i> or <i>Far</i> jump. For <i>near</i> jumps, only IP is reloaded. For <i>far</i> jumps, both CS and IP are changed. (Clock cycles change depending on intra/inter segment)	11-18	2-5	JMP NEXT; (direct jump) JMP BX; indirect near jump JMP DWORD PTR [SI]; indirect far jump.
JAE/JNB/ JNC Label	Jump if Above or Equal/Jump if Not Below/Jump if No Carry;	16/4	2	Jump if CF = 0
JB / JNAE /JC Label	Jump if Below/ Jump if Not Above or Equal/ Jump if Carry	16/4	2	Jump if CF = 1
JNE / JNZ Label	Jump if Not Equal / Jump if Not Zero;	16/4	2	Jump if ZF = 0
JE / JZ Label	Jump if Equal/ Jump if Zero;	16/4	2	Jump if ZF = 1
JNP/JPO Label	Jump if No Parity / Jump if Odd Parity;	16/4	2	Jump if PF = 0
JP/ JPE Label	Jump if Parity / Jump if Even Parity;	16/4	2	Jump if PF = 1
JNO Label	Jump if No Overflow;	16/4	2	Jump if OF = 0
JO Label	Jump if Overflow;	16/4	2	Jump if OF = 1
JNS Label	Jump if Not Signed; (Jump if Positive)	16/4	2	Jump if SF = 0

Conditional Jump Table contd...

JS Label	Jump if Signed; (Jump if negative)	16/4	2	Jump if SF = 1
JA/ JNBE Label	Jump if Above/ Jump if Not Below or Equal;	16/4	2	Jump to address Label, if CF = ZF = 0.
JBE/JNA Label	Jump if Below or Equal/ Jump if Not Above	16/4	2	Jump if CF = 1 or ZF = 1
JG/JNLE Label	Jump if Greater/ Jump if Not Less or Equal	16/4	2	Jump if ZF = 0 and CF = OF
JGE/JNL Label	Jump if Greater or Equal/ Jump if Not Less	16/4	2	Jump if SF = OF
JL / JNGE Label	Jump if Less / Jump if Greater or Equal	16/4	2	Jump if SF ≠ OF
JLE / JNG	Jump if Less / Jump if Not Greater	16/4	2	Jump if ZF = 1, or SF ≠ OF
JCXZ Label	Jump if CX register is Zero	18/6	2	
CALL Label (can be near or far)	Call a Subroutine;			CALL MULT; (near -direct)
	Intrasegment (near) direct	19	3	CALL BX; (near - indirect)
	Intrasegment indirect through register	16	2	CALL WORD PTR [BX];
	Intrasegment indirect through memory	21 + EA	2-4	(near-indirect)
	Intersegment (far) direct	28	5	CALL DWORD PTR [BX];
	Intersegment indirect	37 + EA	2-4	(far – indirect)
RET	Return from a procedure;			
	Intrasegment	8	1	After retrieving CS and IP, increment SP by the const value
	Intrasegment with constant	12	3	
	Intersegment	18	1	
	Intersegment with constant	17	3	

String manipulation instructions

- MOVS/ MOVSB/ MOVSW
- Transfers a byte/word/double word from the data segment addressed by SI to the extra segment addressed by DI
- The pointer then increments depending on the status of directional flag
- If D=0, pointer is incremented, D=1, it is decremented.
- Examples
- MOVSB $ES:[DI] \leftarrow DS:[SI]$ $DI=DI\pm 1, SI=SI\pm 1$
- MOVSW $ES:[DI] \leftarrow DS:[SI]$ $DI=DI\pm 2, SI=SI\pm 2$
- Application: Video Text display
- **Examples**
- MOV SI, OFFSET SOURCE Load offset of start of source string in DS into SI
- MOV DI, OFFSET DESTINATION Load offset of start of destination string in ES into DI
- CLD Clear DF to auto increment SI and DI after move
- MOV CX, 04H Load length of string into CX as counter
- REP MOVSB Move string byte until CX = 0

- MOV SI, OFFSET SOURCE Load offset of start of source string in DS into SI
- MOV DI, OFFSET DESTINATION Load offset of start of destination string in ES into DI
- CLD Clear DF to auto increment SI and DI after move
- MOV CX, 04H Load length of string into CX as counter
- REP MOVSB Move string byte until CX = 0

Example: To move a string of length 4 bytes from SRC to DST

- MOV SI, offset SRC
- MOV DI, offset DST
- MOV CX, 04H
- CLD; Clear the direction flag
- REP MOVSB

LODS / LODSB / LODSW (LOAD STRING BYTE INTO AL OR STRING WORD INTO AX)

- This instruction copies a byte from a string location pointed to by SI to AL, or a word from a string location pointed to by SI to AX.
- If D=0, SI will be automatically incremented (by 1 for a byte string, and 2 for a word string) to point to the next element of the string.
- If D=1, SI will be automatically decremented (by 1 for a byte string, and 2 for a word string) to point to the previous element of the string. LODS does not affect any flag.
- **Examples**
 - CLD ; Clear direction flag so that SI is auto-incremented
 - MOV SI, OFFSET SOURCE ;Point SI to start of string
 - LODS SOURCE; Copy a byte or a word from string to AL or AX

STOS / STOSB / STOSW (STORE STRING BYTE OR STRING WORD)

- This instruction copies a byte from AL or a word from AX to a memory location in the extra segment pointed to by DI.
- In effect, it replaces a string element with a byte from AL or a word from AX.
- After the copy, DI is automatically incremented or decremented to point to next or previous element of the string.
- If DF is cleared, then DI will automatically incremented by 1 for a byte string and by 2 for a word string.
- If DI is set, DI will be automatically decremented by 1 for a byte string and by 2 for a word string. STOS does not affect any flag.
- MOV DI, OFFSET TARGET
- STOSB TARGET

Convert all string characters to lower case using load and store string instructions

MOV SI, offset Array

MOV DI, SI

MOV CX, 100 ; Array length

MOV AX, DS

MOV ES, AX

CLD

Next: LODSB

OR AL, 20H ; convert to lower case

STOSB

LOOP Next

'a' - 97d = 61h , 'A' = 65d = 41h

if the character is already an upper case, it will remain upper case by the instruction OR AL, 20H. However, the lower case characters will be converted to upper case characters.

Example

- The **store** string instructions are useful in initializing a block of memory with a particular value.

```
MOV DI, offset Array
      MOV CX, 100
      MOV AX, DS
      MOV ES, AX
      CLD
      MOV AL, -1
REP  STOSB
```

CMPS / CMPSB / CMPSW (COMPARE STRING BYTES OR STRING WORDS)

- This instruction can be used to compare a byte / word in one string with a byte / word in another string.
- SI is used to hold the offset of the byte or word in the source string, and DI is used to hold the offset of the byte or word in the destination string.
- The AF, CF, OF, PF, SF, and ZF flags are affected by the comparison, but the two operands are not affected.
- After the comparison, SI and DI will automatically be incremented or decremented to point to the next or previous element in the two strings.
- If DF is set, then SI and DI will automatically be decremented by 1 for a byte string and by 2 for a word string.
- If DF is reset, then SI and DI will automatically be incremented by 1 for byte strings and by 2 for word strings.
- The string pointed to by SI must be in the data segment.
- The string pointed to by DI must be in the extra segment.

Example

- Compare, till a difference is found, two data items of 100 bytes each at STR1 and STR2
- MOV SI, STR1
- MOV DI, STR2
- MOV CX, 100
- CLD; Clear the direction flag
- REPE CMPSB

SCAS / SCASB / SCASW (SCAN A STRING BYTE OR A STRING WORD)

- SCAS compares a byte in AL or a word in AX with a byte or a word in ES pointed to by DI.
- Therefore, the string to be scanned must be in the extra segment, and DI must contain the offset of the byte or the word to be compared.
- If DF is cleared, then DI will be incremented by 1 for byte strings and by 2 for word strings.
- If DF is set, then DI will be decremented by 1 for byte strings and by 2 for word strings.
- SCAS affects AF, CF, OF, PF, SF, and ZF, but it does not change either the operand in AL (AX) or the operand in the string.
- *REPNE SCASW Scan a string of word until a word in the string matches the word in AX or until all of the string has been scanned*

Example program using SCASB

- **MOV AX, 2000h** ;move data to register AX.
- **MOV ES, AX** ; move the data of AX to segment register ES.
- **MOV DI, 600h** ; move offset 600 to Destination Index(DI).
- **MOV AX, 25h** ; move data to AL.
- **MOV CX, 0005** ;move data to CX.
- **CLD** ; clear the value of Directional Flag (DF),
- **REPNE SCASB** ; scan data from DI and compare it with data from AL, if equal go to next step else increment the value of DI by 1 and repeat this step.
- **HLT** stops executing the program and halts any further execution.

Examples of Program execution transfer instructions

- `CMP AX, 4371H` ;Compare by subtracting 4371H from AX
- `JA NEXT` ;Jump to label NEXT if AX above 4371H
- `CMP AX, 4371H` ;Compare (AX – 4371H)
- `JNBE NEXT` ;Jump to label NEXT if AX not below or equal to 4371H

Examples of Program execution transfer instructions

- `CMP AX, 4371H` Compare (AX – 4371H)
- `JAE NEXT` Jump to label NEXT if AX above 4371H
- `CMP AX, 4371H` Compare (AX – 4371H)
- `JNB NEXT` Jump to label NEXT if AX not below 4371H
- `ADD AL, BL` Add two bytes
- `JNC NEXT` If the result with in acceptable range, continue

Example

- `CMP AX, 4371H` Compare (AX – 4371H)
- `JB NEXT` Jump to label NEXT if AX below 4371H
- `ADD BX, CX` Add two words
- `JC NEXT` Jump to label NEXT if CF = 1
- `CMP AX, 4371H` Compare (AX – 4371H)
- `JNAE NEXT` Jump to label NEXT if AX not above or equal to 4371H

Examples of Program execution transfer instructions

- **JO (JUMP IF OVERFLOW)**
- The overflow flag will be set if the magnitude of the result produced by some signed arithmetic operation is too large to fit in the destination register or memory location. The JO instruction will cause a jump to the destination given in the instruction, if the overflow flag is set.
- **ADD AL, BL** Add signed bytes in AL and BL
- **JO ERROR** Jump to label ERROR if overflow from add

Loop (Jump to Specified Label If CX = 0 after auto decrement)

- This instruction is used to repeat a series of instructions some number of times.
- The number of times the instruction sequence is to be repeated is loaded into CX.
- Each time the LOOP instruction executes, CX is automatically decremented by 1.
- If CX is not 0, execution will jump to a destination specified by a label in the instruction.
- If CX = 0 after the auto decrement, execution will simply go on to the next instruction after LOOP.
- The destination address for the jump must be in the range of -128 bytes to +127 bytes from the address of the instruction after the LOOP instruction. This instruction does not affect any flag.

Example

- `MOV BX, OFFSET PRICES`; Point BX at first element in array
- `MOV CX, 40` ;Load CX with number of elements in array
- `NEXT: MOV AL, [BX]`; Get element from array
- `INC AL` ;Increment the content of AL
- `MOV [BX], AL` ;Put result back in array
- `INC BX` ;Increment BX to point to next location
- `LOOP NEXT` ;Repeat until all elements adjusted

LOOPE / LOOPZ (LOOP WHILE CX = 0 AND ZF = 1)

- This instruction is used to repeat a group of instructions some number of times, or until the zero flag becomes 0.
- The number of times the instruction sequence is to be repeated is loaded into CX.
- Each time the LOOP instruction executes, CX is automatically decremented by 1.
- If CX = 0 and ZF = 1, execution will jump to a destination specified by a label in the instruction.
- If CX = 0, execution simply go on the next instruction after LOOPE / LOOPZ.
- In other words, the two ways to exit the loop are CX = 0 or ZF = 0. The destination address for the jump must be in the range of -128 bytes to +127 bytes from the address of the instruction after the LOOPE / LOOPZ instruction. This instruction does not affect any flag.

Example

- `MOV BX, OFFSET ARRAY ;Point BX to address of ARRAY`
- `DEC BX ;Decrement BX`
- `MOV CX, 100 ;Put number of array elements in CX`
- `NEXT: INC BX ;Point to next element in array`
- `CMP [BX], 0FFH ;Compare array element with FFH`
- `LOOPE NEXT`

Unit II: Logical Instructions and Control/Branch Instructions

Logical Instructions, String manipulation instructions, Flag manipulation and Processor control instructions, Illustration of these instructions with example programs, control and branch instructions. Assembler Directives and Operators, Assembly Language Programming and example programs

Assembly language

- Assembly languages are low-level languages for programming computers, microprocessors, microcontrollers, and other IC.
- They implement a symbolic representation of the numeric machine Codes and other constants needed to program a particular CPU architecture.
- This representation is usually defined by the hardware manufacturer, and is based on abbreviations that help the programmer to remember individual instructions, registers.
- An **assembler directive** is a statement to give direction to the assembler to perform task of the assembly process.

Assembler Details

High-level Language

```
temp  = v[k];  
v[k]  = v[k+1];  
v[k+1] = temp;
```

```
TEMP = V(K)  
V(K)  = V(K+1)  
V(K+1) = TEMP
```

C/Java Compiler

Fortran Compiler

Assembly Language

```
lw  $t0, 0($2)  
lw  $t1, 4($2)  
sw  $t1, 0($2)  
sw  $t0, 4($2)
```

MIPS Assembler

Machine Language

```
0000 1001 1100 0110 1010 1111 0101 1000  
1010 1111 0101 1000 0000 1001 1100 0110  
1100 0110 1010 1111 0101 1000 0000 1001  
0101 1000 0000 1001 1100 0110 1010 1111
```

Assembler

- Assembler for the microprocessor can be used in 2 ways
 1. With models that are unique to a particular assembler
 2. With full segment definitions that allow compiler to control over the assembly process and are universal to all assemblers
- With `.model tiny` , CS, DS, and SS are all pointing to the same 64KB of memory.
- The stack is placed in the highest region of this 64KB segment
- With `.model small` CS points to a segment of its own, followed by the segment where DS and SS are pointing to. (64k bytes each)
- The stack is placed in the highest region of the `ss` segment.

Assembler Directives

- Indicate how an operand or a section of a program is to be processed by the assembler
- Some directives generate and store information in the memory, while others do not
- Ex: The DB directive stores bytes of data in the memory while BYTE PTR directive never stores data
- The BYTE PTR directive indicates the size of the data referenced by a pointer or index register
- Assembler directives are the commands to the assembler
- They are not instructions to the microprocessor
- These are nothing but pseudo mnemonics
- These directives enable us to control the way a program is assembled and listed
- They do not generate any machine code for execution

Assembler Directives contd...

- **Model directive:**
- The model directive specifies the total amount of memory the program would take.
- i.e, it gives information on how much memory the assembler would allocate for the program.
- This depends on the size of the data and the size of the program or code.
- Example

`.model small`

ASSUME Directive

- The ASSUME directive is used to tell the assembler that the name of the logical segment should be used for a specified segment.
- The 8086 works directly with only 4 physical segments: a Code segment, a data segment, a stack segment, and an extra segment.
- **Example:**
- **ASUME CS:CODE** ;This tells the assembler that the logical segment named CODE contains the instruction statements for the program and should be treated as a code segment.
- **ASUME DS:DATA** ;This tells the assembler that for any instruction which refers to a data in the data segment, data will found in the logical segment DATA

ENDS Directive

- This directive is used with the name of a segment to indicate the end of that logical segment
- ENDS is used with the segment directive to 'bracket' a logical segment containing instructions or data
- Ex1: ASSUME CS: Code_here

...

...

Code_here ENDS

Ex2: ASSUME DS:DATA

num1 db 20h

Num 2 db 45h

Res db ??

DATA ENDS

END Directive

- END Directive: END program: The END directive is put after the last statement of a program to tell the assembler that this is the end of the program module
- ENDP-End Procedure: This is used along with the name of the procedure to indicate the end of a procedure to the assembler
- This directive together with the procedure directive PROC is used to bracket a procedure
- Ex: sqrt_root PROC
.....
.....
- sqrt_root ENDP

DB directive

- DB-Define Byte -DB directive is used to declare a byte type variable or to store a byte in memory location
- Examples
- NUM DB 42h tells the assembler to declare a byte named NUM and initialize 42h to NUM
- PRICE DB 49h, 98h, 29h ;Declare an array of 3 bytes, named as PRICE and initialize.
- TEMP DB 100 DUP(?) ;Set 100 bytes of storage in memory and give it the name as TEMP, but leave the 100 bytes uninitialized.
- Program instructions will load values into these locations.

DW - The DW directive

- **DW** - The DW directive is used to define a variable of type word or to reserve storage location of type word in memory.
- **Example:**
- **MULTIPLIER DW 437Ah** ; this declares a variable of type word and named it as MULTIPLIER.
- This variable is initialized with the value 437Ah when it is loaded into memory to run.
- **EXP1 DW 1234h, 3456h, 5678h** ; this declares an array of 3 words and initialized with specified values.
- **STOR1 DW 100 DUP(0)**; Reserve an array of 100 words of memory and initialize all words with 0000. Array is named as STOR1.

- **EQU** - This EQU directive is used to give a name to some value or to a symbol. Each time the assembler finds the name in the program, it will replace the name with the value or symbol you given to that name.

Example:

- **FACTOR EQU 03H** ; this statement must be written at the starting of your program and later in the program you can use this as follows
- **ADD AL, FACTOR** ;
- When it codes this instruction the assembler will code it as ADD AL, 03H
- ;The advantage of using EQU in this manner is, if FACTOR is used many no of times in a program and you want to change the value, all you had to do is change the EQU statement at beginning, it will changes the rest of all.

- **PTR** - This **PTR** operator is used to assign a specific type of a variable or to a label.

Example:

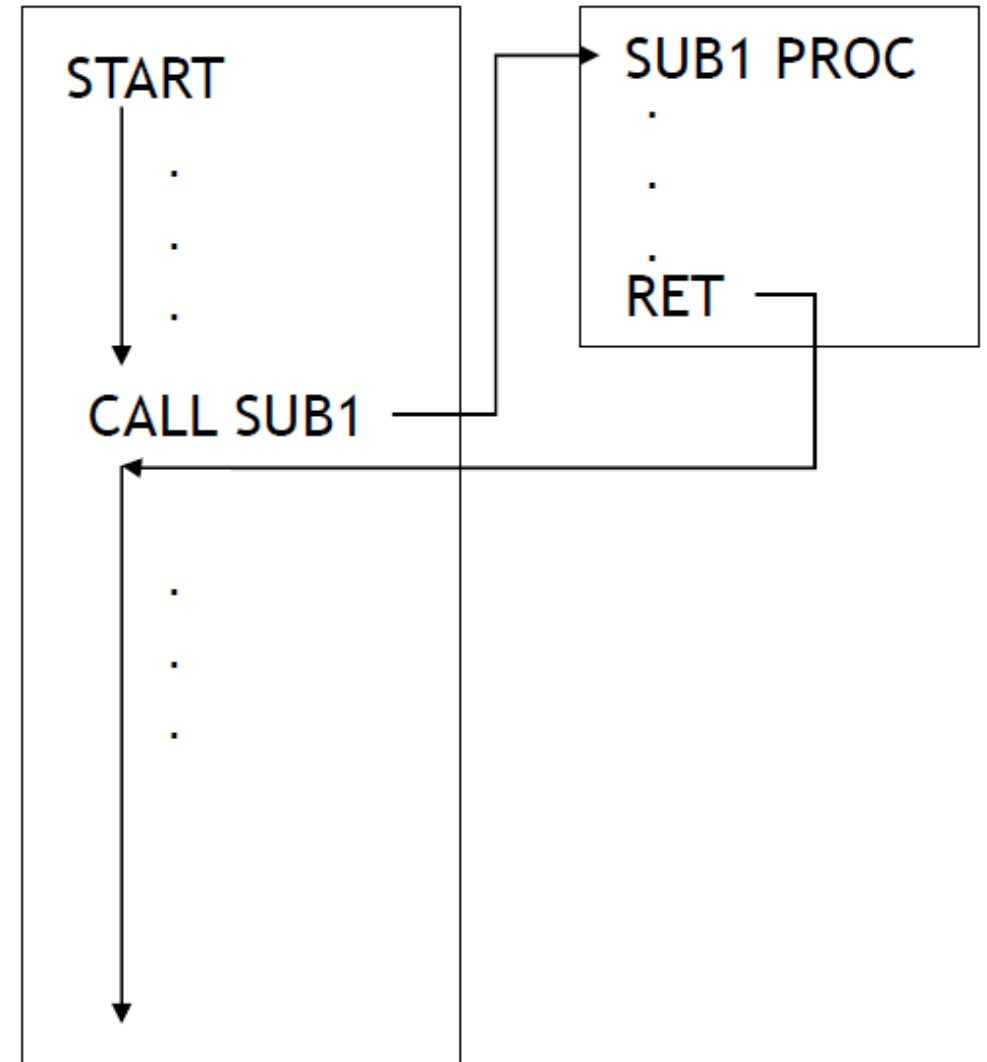
- **INC [BX]** ; This instruction will not know whether to increment the byte pointed to by BX or a word pointed to by BX.
- **INC BYTE PTR [BX]** ;increment the byte pointed to by BX
- This PTR operator can also be used to override the declared type of variable .
- If we want to access the a byte in an array **WORDS DW 437Ah, 0B97h,**
- **MOV AL, BYTE PTR WORDS**

Example Program

- Q1. Byte and word move using different addressing modes
- Program:
- .model small
- .data
- Num dw 4321h
- .code
- Mov ax,@data
- Mov ds,ax
- Mov ax,1234h ;immediate addressing
- Mov bx,ax ;register addressing
- Mov ax,num ;direct addressing
- Mov si,1000h
- Mov al,[si] ;indirect addressing
- Mov bl,[si+100h] ;relative addressing
- Mov bx,1000h
- Mov ax,[si+bx] ;base index addressing
- Mov cx,[si+bx+100h] ;relative base index addressing
- Mov ah,4ch
- Int 21h
- End
- Output:
- Ax=1234h
- Bx=1234h
- Ax=4321h.....
- <https://cnx.org/contents/7M5NhNIS@1/Microprocessor-8086-Lab>

Procedure

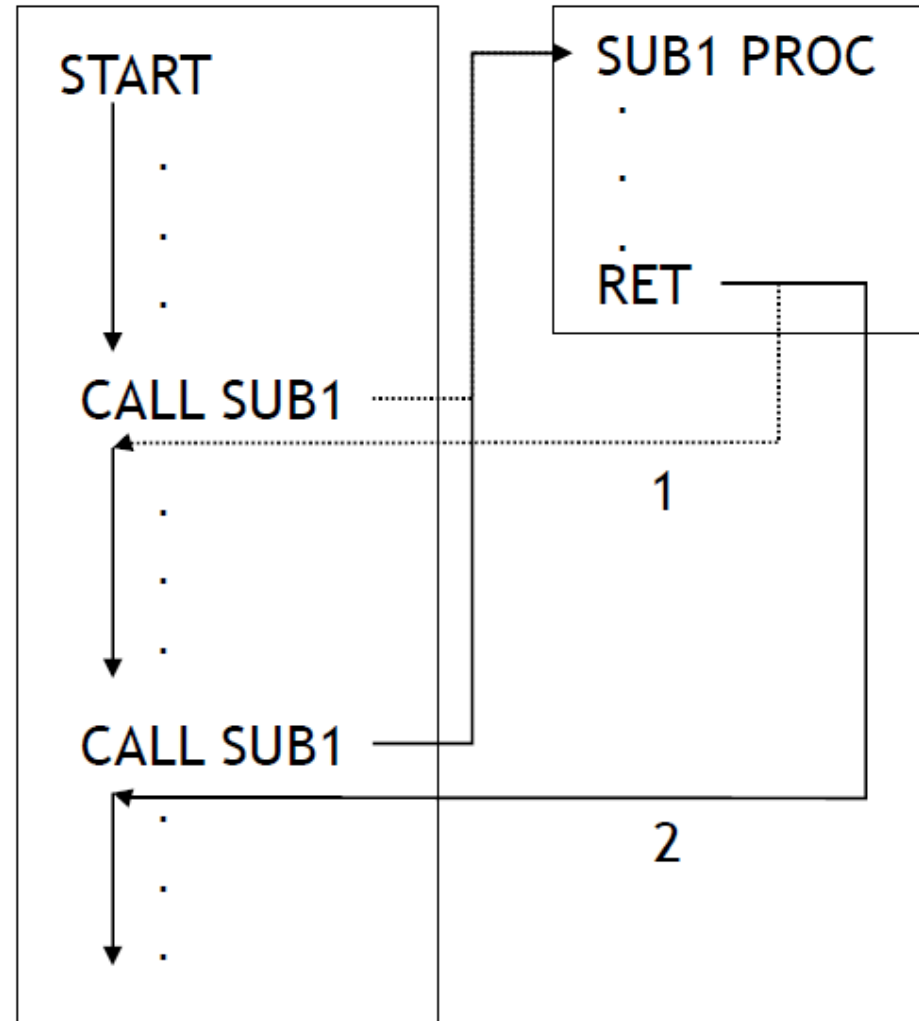
- It is a group of instructions that usually performs one task.
- It is a reusable section of the software that is stored in memory once but used as often as necessary
- This saves memory space and makes it easier to develop software
- The only disadvantage of a procedure is that it takes the computer a small amount of time to link to the procedure and return from the procedure
- The CALL instruction links to the procedure and the RET instruction returns from the procedure



Procedure contd..

- The stack stores the **return address** whenever a procedure is called during execution of a program
- The CALL instruction **pushes the address of the instruction following the CALL** on the stack, RET **removes the address from the stack** so the program returns to the instruction following the CALL
- A procedure begins with the **PROC directive** and ends with the **ENDP directive**.
- Near procedure removes 16 bit address from stack and places into IP to return from the procedure in the current code segment
- A far procedure removes a 32 bit number from the stack and places it into IP and CS to return from the procedure to any memory location
- Procedures that are to be used by all software (global) should be written as far procedure
- Procedure that is to be used by a given task (local) is normally defined as near procedure

Procedure call using CALL instruction



Syntax

Procedure name PROC near
instruction 1
instruction 2
RET

Procedure name ENDP

Example:

ADD2 PROC near
ADD AX,BX
RET
ADD2 ENDP

```
.model small
.data
str1 db "Hello$"
str2 db "How are
you$"
str3 db "I am fine $"
.code
mov ax,@data
mov ds, ax
mov dx, offset str1
mov ah, 09h
int 21h
call enterkey
mov dx, offset str2
mov ah,09h
int 21h
call enterkey
mov dx, offset str3
mov ah,09h
int 21h
call enterkey
mov ah, 4ch
int 21h
```

```
enterkey proc near
mov dx,10
mov ah, 02h
int 21h
mov dx, 13
mov ah,02
int 21h
ret
enterkey endp
end
```

Assembly language example programs

.model small

.data

N1 db "Enter n1\$"

N2 db "Enter n2\$"

Res db 2 dup ??

.code

Mov ax, @data

Mov ds, ax

Mov ah, 01h

Int 21h

Mov bl, al

Mov ah, 01h

Int 21h

Add al, bl

Mov res, al

Mov ah, 4ch

Int 21h

end

Mov dx, offset n1

Mov ah, 09h

Int 21h

Will display Enter n1 on
command prompt

Far Procedure

A far procedure refers to a procedure which is in the different code segment from that of the call instruction (program)

2. It is also called inter-segment procedure call
3. A far procedure call replaces the old CS:IP pairs with new CS:IP pairs
4. The value of the old CS:IP pairs are pushed on to the stack $SP=SP-2$
5. More stack locations are required
6. Procedure is outside Code Segment
7. Uses keyword FAR
8. Example :- Call FAR PROC Delay
9. Syntax:- CALL far_procedure_name

Any errors in the code?

.model small

.code

One proc near

Mov ah,02h

Mov dl, 'A'

Int 21h

ret

One proc endp

end

MACROs

- **MACRO** is a group of instructions with a name.
- When a macro is invoked, the associated set of instructions is **inserted in place** where the MACRO is called, replacing the macro name.
- A macro can be defined anywhere in program using the directives **MACRO** and **ENDM**
- When a macro is invoked, the corresponding code is inserted into the source wherever macro name appears in the program.
- Thus multiple copies of the same code exist in the memory and so large memory space is needed.

Examples of MACRO definition and usage

- Examples of Macro Definitions:
 - ; Definition of a Macro named addition
 - addition MACRO**
 - MOV AX, 1344H ; copy a number into AX**
 - MOV BX, 3435H**
 - ADD AX, BX**
 - ENDM ; indicates end of macro definition**
- Examples of Macro usage:
- The following examples illustrate the use of macros. We first show the source with macro invocation and then show how the expanded source looks.
- Program with macro invocations:
 - Main program
 -
 - MOV CX, 33
 - addition ; invoke macro using the name
 - ADD AX, CX

Examples of MACRO continued

- When the Macro Preprocessor expands the MACROs in the previous source, the expanded source looks as shown below:

MOV CX, 33

MOV AX, 1344H

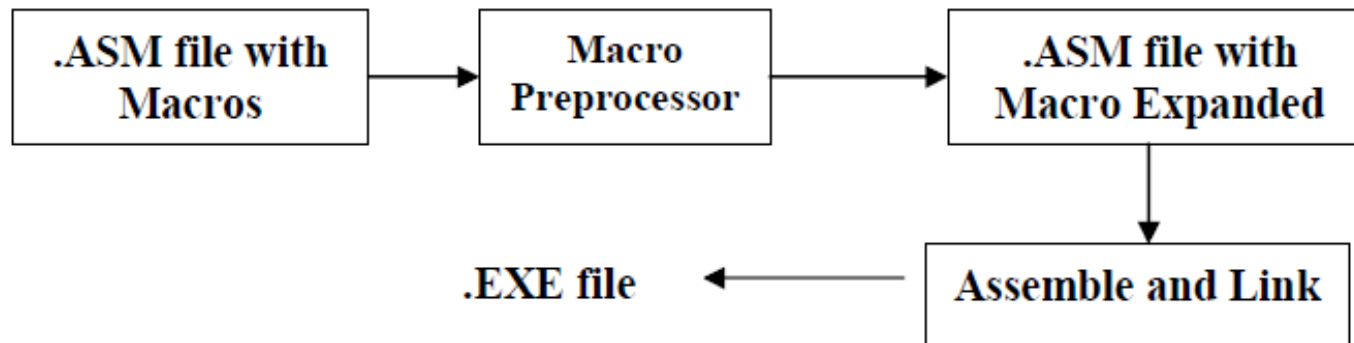
MOV BX, 3435H

ADD AX, BX

ADD AX, CX ; final sum in AX = 1344h+3435h+0033h

MACROS continued...

- Note how the macro name is replaced by the associated set of instructions.
- Thus, **macro name does not appear in the expanded source code.**
- In other words, the actual Assembler does not see the macros.
- What gets assembled is the expanded source.
- Body of the macro gets substituted at the place where the macro name appears in the program.



Procedure vs Macro

Procedures	Macros
Accessed by CALL and RET mechanism during program execution	Accessed by name given to macro when defined during assembly
Machine code for instructions only put in memory once	Machine code generated for instructions each time called
Parameters are passed in registers, memory locations or stack	Parameters passed as part of statement which calls macro
Procedures uses stack	Macro does not utilize stack
A procedure can be defined anywhere in program using the directives PROC and ENDP	A macro can be defined anywhere in program using the directives MACRO and ENDM
Procedures takes huge memory for CALL(3 bytes each time CALL is used) instruction	Length of code is very huge if macro's are called for more number of times

Procedure vs Macro

Procedures	Macros
1. Use PROC and ENDP directives 2. Stored only once in the memory , hence take less memory 3. Control is transferred to the place where procedure is present, hence more time needed for execution 4. Makes use of stack to store the return address 5. Difficult to pass parameters 6. CALL instruction is used to call a procedure	1. Use MACRO and ENDM directives 2. Instructions replace all invocations; hence more memory space needed 3. Since, instructions are copied at all invocations, no need to transfer the control, hence comparatively less time is needed 4. No need of stack as no control transfer 5. Easy to pass parameters 6. No instruction is needed to invoke a macro